Mini project report on

Operating System

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE24CS242B - Operating Systems

Submitted by:

Prajwal M	**PES2UG24CS910**
R P Pranav	**PES2UG24CS917**

under the guidance of

Prof. Neha Sharma

Assistant Professor

PES University

Jan- May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Operating Systems

is a bonafide work carried out by

Prajwal M	**PES2UG24CS910**
R P Pranav	**PES2UG24CS917**

In partial fulfilment for the completion of forth semester OS Project (UE24CS242B) in the Program of Study - Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period Jan 2026 – May 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 4th semester academic requirements in respect of project work.

Signature

Prof. Neha Sharma
Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **Operating System** has been carried out by us under the guidance of **Prof. Neha Sharma, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester Jan – May 2026.

Prajwal M	**PES2UG24CS910**
R P Pranav	**PES2UG24CS917**

Prajwal M	**PES2UG24CS910**
R P Pranav	**PES2UG24CS917**

ACKNOWLEDGEMENT

I would like to express my gratitude to Prof. Neha Sharma, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS242B - Operating Systems Project.

I take this opportunity to thank Dr. Sandesh B J, C, Professor, Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	1
2.	PROBLEM DEFINITION	4
3.	METHODOLOGY	8
4.	CODE	12
5.	RESULT AND DISCUSSION	18
6.	OUTPUTS	20
7.	GITHUB LINK	21
REFERENCES/BIBLIOGRAPHY		22
APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS		23

This project implements a lightweight Linux container runtime in C, comprising a long-running usr-space supervisor (engine.c) and a kernel-space memory monitoring module (monitor.c). The runtime provides multi-container isolation using Linux namespaces (PID, UTS, and mount) and chroot, enabling each container to operate in a sandboxed environment with an Alpine Linux root filesystem. The supervisor manages container lifecycles, handles concurrent log capture via a bounded-buffer producer-consumer pipeline, and exposes a CLI for commands such as start, run, ps, logs, and stop. A UNIX domain socket serves as the IPC channel between the CLI client and the supervisor. A Linux Kernel Module tracks monitored container processes, enforces soft memory limits by writing kernel log warnings, and enforces hard limits by terminating offending processes. Scheduler experiments comparing CPU-bound and I/O-bound workloads under different nice values demonstrate observable differences in CPU share consistent with the Linux Completely Fair Scheduler (CFS). The project exercises core OS concepts including process isolation, IPC, synchronization, kernel module development, and scheduling.

CHAPTER 1
INTRODUCTION

Containerization has become a foundational technology in modern software deployment, enabling applications to run in isolated environments without the overhead of full virtual machines. At the core of any container system lie the operating system's primitive mechanisms: namespaces for isolation, cgroups for resource control, and the kernel's process management facilities. This project explores these mechanisms by building a lightweight container runtime from scratch in C, targeting Linux systems running Ubuntu 22.04 or 24.04.

The project is named Jackfruit and is developed as part of UE24CS242B - Operating Systems at PES University. It consists of two integrated components: a user-space runtime and supervisor process (engine.c) that launches and manages multiple isolated containers simultaneously, maintains per-container metadata, provides a command-line interface, and captures container output through a concurrent logging pipeline; and a Linux Kernel Module (monitor.c) that tracks container processes, enforces configurable memory limits, and integrates with the user-space runtime via ioctl calls.

Linux namespaces form the foundation of container isolation. The PID namespace ensures processes inside a container see only their own process tree. The UTS namespace allows each container to have its own hostname. The mount namespace provides an isolated filesystem view using chroot with an Alpine Linux root filesystem. The host kernel remains shared across all containers, which is a fundamental distinction between containerization and full hardware virtualization.

Beyond isolation, this project addresses concurrent programming challenges inherent in managing multiple containers. The supervisor handles signals (SIGCHLD, SIGINT, SIGTERM), reaps zombie child processes, synchronizes access to shared metadata, and implements a bounded-buffer logging system where container stdout and stderr serve as producer streams and logging threads act as consumers writing to per-container log files.

The kernel module extends the runtime with policy-based memory enforcement. By tracking container host PIDs in a kernel-maintained linked list protected by a mutex, the module periodically samples each process's Resident Set Size (RSS) and applies either a soft-limit warning or a hard-limit SIGKILL

depending on the configured threshold. This is more reliable than user-space polling since kernel-space enforcement cannot be bypassed by the monitored process.

The project also includes controlled scheduling experiments that use the runtime as an experimental platform to observe how the Linux Completely Fair Scheduler (CFS) treats CPU-bound and I/O-bound workloads under different scheduling configurations, providing empirical evidence of scheduler fairness, responsiveness, and throughput characteristics.

CHAPTER 2
PROBLEM DEFINITION

The problem this project addresses is the design and implementation of a production-inspired Linux container runtime that exercises all major subsystems of a modern operating system: process management, inter-process communication, synchronization, memory management, kernel programming, and scheduling. Existing tools such as Docker abstract these mechanisms behind high-level interfaces. Building a runtime from scratch requires direct engagement with the underlying OS primitives.

2.1 Core Functional Requirements

The runtime must support multiple simultaneous containers, each running in isolated PID, UTS, and mount namespaces using an Alpine Linux root filesystem. A long-running parent supervisor must track metadata for each container including its name, host PID, start time, current state (starting, running, stopped, or killed), configured memory limits, log file path, and exit status. All shared metadata access must be thread-safe.

A command-line interface must allow users to interact with the supervisor at runtime. Required commands include: start (launch a container in the background), run (launch and wait in the foreground), ps (list all tracked containers and their metadata), logs (view captured container output), and stop (terminate a container gracefully). The CLI client communicates with the supervisor via a UNIX domain socket, which is a separate IPC channel from the logging pipeline.

Container output must be captured through a concurrent logging pipeline. Each container's stdout and stderr are connected to the supervisor via pipes. The supervisor maintains a bounded circular buffer where producer threads insert log chunks and a consumer thread writes them to persistent per-container log files. The pipeline must be correctly synchronized to prevent race conditions, data loss, corruption, and deadlock.

2.2 Kernel Module Requirements

The kernel module must register a control device at /dev/container_monitor, accept host PIDs from the supervisor via ioctl (with soft and hard memory limit parameters), and periodically monitor each registered process's RSS. When a process exceeds the soft limit, a warning is written to the kernel log. When it exceeds the hard limit, the module sends SIGKILL and the supervisor updates the container's state to reflect the kill reason. On module unload, all kernel list entries must be freed and the device must be unregistered cleanly.

2.3 Engineering Challenges

Signal handling is complex in a multi-container supervisor: SIGCHLD must be handled asynchronously without introducing races with the main event loop. Zombie process prevention requires reliable reaping of all children. The bounded-buffer design must handle multiple producers (one per container) and one consumer thread, requiring careful use of pthread_mutex_t and pthread_cond_t. Kernel module development requires locking discipline, correct use of kernel APIs, and safe cleanup on unload to prevent memory leaks or kernel panics. Scheduler experiments require careful workload design to produce measurable and reproducible results.

CHAPTER 3
METHODOLOGY

The project is implemented in three phases: user-space runtime development, kernel module development, and experimental evaluation. The development environment is Ubuntu 22.04 or 24.04 in a virtual machine with Secure Boot disabled. Dependencies include the GNU C compiler, make, and Linux kernel headers matching the running kernel version, installed via: sudo apt install build-essential linux-headers-$(uname -r).

3.1 Environment Setup

Before implementation, an environment preflight check script (environment-check.sh) verifies that all dependencies are installed, the kernel supports the required namespace types, and the user has sufficient privileges. The Alpine Linux mini root filesystem is downloaded from the official Alpine CDN and extracted into a local rootfs/ directory, which is not committed to the repository. Workload binaries that must run inside containers are copied into rootfs/ before launching the relevant container.

3.2 Container Isolation and Spawn Mechanism

Each container is created using the clone() system call with flags CLONE_NEWPID, CLONE_NEWUTS, and CLONE_NEWNS combined with SIGCHLD. Inside the child function, the process changes its root directory using chroot() to the Alpine rootfs, changes its working directory to /, mounts /proc as a procfs so process introspection tools function correctly, sets its hostname to the container name using sethostname(), and executes the target binary using execv(). Pipe file descriptors are set up before clone() and the write end is inherited by the container as its stdout and stderr via dup2().

3.3 Supervisor Architecture and Signal Handling

The supervisor is a long-running parent process that maintains a dynamically allocated array of container_t metadata structures protected by a pthread_mutex_t. It initializes a UNIX domain socket server bound to a well-known path (e.g., /tmp/jackfruit.sock) to accept commands from CLI clients. A SIGCHLD handler uses waitpid() in a non-blocking loop with WNOHANG to reap all exited children and updates the corresponding metadata entry's state. SIGINT and SIGTERM handlers set a global

shutdown flag that triggers an orderly teardown: all running containers are stopped, logging threads are signalled and joined, and file descriptors are closed before the supervisor exits.

3.4 Bounded-Buffer Logging Pipeline

For each container a pipe pair is created before clone(); the write end is used by the container and the read end is held by the supervisor. A producer pthread is spawned per container; it reads from the pipe into a fixed-size buffer and inserts the chunk into a shared circular ring buffer. A consumer pthread removes chunks from the ring buffer and appends them to the per-container log file. The ring buffer is protected by a pthread_mutex_t; a pthread_cond_t named not_empty wakes the consumer when data is available, and a pthread_cond_t named not_full blocks producers when the buffer is full. This design avoids busy-waiting and handles clean shutdown when the pipe is closed.

3.5 CLI Design and IPC

The CLI is a separate executable that connects to the supervisor's UNIX domain socket, sends a structured command message (containing the command name and arguments as a null-terminated string), and reads the supervisor's text response. The supervisor's socket server runs in the main loop or a dedicated thread, parses incoming commands, dispatches to the appropriate handler, and writes a response. This two-channel IPC design keeps the logging pipeline (pipes) and the control channel (UNIX socket) independent so they do not interfere with each other.

3.6 Kernel Memory Monitor Design

The kernel module is structured as a standard LKM with init and exit functions. It registers a character device using register_chrdev() or the newer cdev API, and defines an ioctl handler that accepts a monitor_reg_t structure containing a PID, soft limit, and hard limit. Registered entries are stored in a kernel linked list using the list_head API, protected by a DEFINE_MUTEX. A periodic memory check is scheduled using a delayed workqueue (schedule_delayed_work) that fires every two seconds, iterates the list, resolves each PID to a task_struct using find_vpid() and pid_task(), reads the process RSS from get_mm_rss(task->mm), compares against limits, and takes action. Stale entries (where the task no longer exists) are removed and freed during the check.

3.7 Scheduler Experiment Design

Two workload programs are implemented: a CPU-bound workload that computes a fixed number of floating-point operations in a tight loop, and an I/O-bound workload that repeatedly writes and flushes small blocks to a file. Experiments are run by launching containers with these workloads under different configurations and measuring wall-clock completion time using the time command. Configuration variants include: (a) equal nice values for two CPU-bound containers, (b) one CPU-bound container at nice 0 and another at nice +10, and (c) a CPU-bound container and an I/O-bound container at equal priorities. Results are compared to CFS theory predictions.

CHAPTER 4
CODE

The project consists of three primary source files: engine.c (user-space supervisor and runtime), monitor.c (kernel module), and monitor_ioctl.h (shared ioctl definitions). Representative code excerpts illustrating the key design decisions are presented below.

4.1 Shared ioctl Definitions (monitor_ioctl.h)

```
#ifndef MONITOR_IOCTL_H
#define MONITOR_IOCTL_H

#ifdef __KERNEL__
#include <linux/ioctl.h>
#include <linux/types.h>
#else
#include <sys/ioctl.h>
#include <sys/types.h>
#endif

#define MONITOR_NAME_LEN 32

struct monitor_request {
    pid_t pid;
    unsigned long soft_limit_bytes;
    unsigned long hard_limit_bytes;
    char container_id[MONITOR_NAME_LEN];
};

#define MONITOR_MAGIC 'M'
#define MONITOR_REGISTER   _IOW(MONITOR_MAGIC, 1, struct monitor_request)
#define MONITOR_UNREGISTER _IOW(MONITOR_MAGIC, 2, struct monitor_request)

#endif
```

4.2 engine.c

```
#define _GNU_SOURCE
#include <errno.h>
#include <fcntl.h>
#include <limits.h>
#include <pthread.h>
```

```c
#include <sched.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ioctl.h>
#include <sys/mount.h>
#include <sys/socket.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <sys/un.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

#include "monitor_ioctl.h"

/* ----------------------------------------------------------------- */
/* Constants                                                         */
/* ----------------------------------------------------------------- */
#define STACK_SIZE            (1024 * 1024)
#define CONTAINER_ID_LEN      32
#define CONTROL_PATH          "/tmp/mini_runtime.sock"
#define LOG_DIR               "logs"
#define CONTROL_MESSAGE_LEN   512
#define CHILD_COMMAND_LEN     256
#define LOG_CHUNK_SIZE        4096
#define LOG_BUFFER_CAPACITY   32
#define DEFAULT_SOFT_LIMIT    (40UL << 20)   /* 40 MiB */
#define DEFAULT_HARD_LIMIT    (64UL << 20)   /* 64 MiB */

/* ----------------------------------------------------------------- */
/* Types                                                             */
/* ----------------------------------------------------------------- */

typedef enum {
    CMD_SUPERVISOR = 0,
    CMD_START,
    CMD_RUN,
    CMD_PS,
    CMD_LOGS,
    CMD_STOP
} command_kind_t;

typedef enum {
    CONTAINER_STARTING = 0,
    CONTAINER_RUNNING,
    CONTAINER_STOPPED,
    CONTAINER_KILLED,
    CONTAINER_EXITED
```

```c
} container_state_t;

typedef struct container_record {
    char                    id[CONTAINER_ID_LEN];
    pid_t                   host_pid;
    time_t                  started_at;
    container_state_t       state;
    unsigned long           soft_limit_bytes;
    unsigned long           hard_limit_bytes;
    int                     exit_code;
    int                     exit_signal;
    int                     stop_requested;   /* set before SIGTERM in stop */
    char                    log_path[PATH_MAX];
    struct container_record *next;
} container_record_t;

typedef struct {
    char   container_id[CONTAINER_ID_LEN];
    size_t length;
    char   data[LOG_CHUNK_SIZE];
} log_item_t;

typedef struct {
    log_item_t      items[LOG_BUFFER_CAPACITY];
    size_t          head;
    size_t          tail;
    size_t          count;
    int             shutting_down;
    pthread_mutex_t mutex;
    pthread_cond_t  not_empty;
    pthread_cond_t  not_full;
} bounded_buffer_t;

typedef struct {
    command_kind_t kind;
    char           container_id[CONTAINER_ID_LEN];
    char           rootfs[PATH_MAX];
    char           command[CHILD_COMMAND_LEN];
    unsigned long  soft_limit_bytes;
    unsigned long  hard_limit_bytes;
    int            nice_value;
} control_request_t;

typedef struct {
    int  status;
    char message[CONTROL_MESSAGE_LEN];
} control_response_t;

typedef struct {
    char id[CONTAINER_ID_LEN];
```

```c
    char rootfs[PATH_MAX];
    char command[CHILD_COMMAND_LEN];
    int  nice_value;
    int  log_write_fd;   /* write-end of the logging pipe */
} child_config_t;

typedef struct {
    int             server_fd;
    int             monitor_fd;
    volatile int    should_stop;
    pthread_t       logger_thread;
    bounded_buffer_t log_buffer;
    pthread_mutex_t  metadata_lock;
    container_record_t *containers;
} supervisor_ctx_t;

/* ---------------------------------------------------------------- */
/* Globals (supervisor side only)                                   */
/* ---------------------------------------------------------------- */
static supervisor_ctx_t *g_ctx = NULL;  /* set once in run_supervisor */

/* ---------------------------------------------------------------- */
/* Usage                                                            */
/* ---------------------------------------------------------------- */
static void usage(const char *prog)
{
    fprintf(stderr,
            "Usage:\n"
            "  %s supervisor <base-rootfs>\n"
            "  %s start <id> <container-rootfs> <command> [--soft-mib N] [--hard-mib N] [--
nice N]\n"
            "  %s run   <id> <container-rootfs> <command> [--soft-mib N] [--hard-mib N] [--
nice N]\n"
            "  %s ps\n"
            "  %s logs <id>\n"
            "  %s stop <id>\n",
            prog, prog, prog, prog, prog, prog);
}

/* ---------------------------------------------------------------- */
/* Argument parsing helpers                                         */
/* ---------------------------------------------------------------- */
static int parse_mib_flag(const char *flag,
                          const char *value,
                          unsigned long *target_bytes)
{
    char *end = NULL;
    unsigned long mib;

    errno = 0;
```

```c
    mib = strtoul(value, &end, 10);
    if (errno != 0 || end == value || *end != '\0') {
        fprintf(stderr, "Invalid value for %s: %s\n", flag, value);
        return -1;
    }
    if (mib > ULONG_MAX / (1UL << 20)) {
        fprintf(stderr, "Value for %s is too large: %s\n", flag, value);
        return -1;
    }
    *target_bytes = mib * (1UL << 20);
    return 0;
}

static int parse_optional_flags(control_request_t *req,
                                int argc, char *argv[],
                                int start_index)
{
    int i;
    for (i = start_index; i < argc; i += 2) {
        char *end = NULL;
        long nice_value;

        if (i + 1 >= argc) {
            fprintf(stderr, "Missing value for option: %s\n", argv[i]);
            return -1;
        }

        if (strcmp(argv[i], "--soft-mib") == 0) {
            if (parse_mib_flag("--soft-mib", argv[i + 1],
                               &req->soft_limit_bytes) != 0)
                return -1;
            continue;
        }
        if (strcmp(argv[i], "--hard-mib") == 0) {
            if (parse_mib_flag("--hard-mib", argv[i + 1],
                               &req->hard_limit_bytes) != 0)
                return -1;
            continue;
        }
        if (strcmp(argv[i], "--nice") == 0) {
            errno = 0;
            nice_value = strtol(argv[i + 1], &end, 10);
            if (errno != 0 || end == argv[i + 1] || *end != '\0' ||
                nice_value < -20 || nice_value > 19) {
                fprintf(stderr,
                        "Invalid value for --nice (expected -20..19): %s\n",
                        argv[i + 1]);
                return -1;
            }
            req->nice_value = (int)nice_value;
```

```
            continue;
        }
        fprintf(stderr, "Unknown option: %s\n", argv[i]);
        return -1;
    }

    if (req->soft_limit_bytes > req->hard_limit_bytes) {
        fprintf(stderr, "Invalid limits: soft limit cannot exceed hard limit\n");
        return -1;
    }
    return 0;
}

static const char *state_to_string(container_state_t state)
{
    switch (state) {
    case CONTAINER_STARTING: return "starting";
    case CONTAINER_RUNNING:  return "running";
    case CONTAINER_STOPPED:  return "stopped";
    case CONTAINER_KILLED:   return "killed";
    case CONTAINER_EXITED:   return "exited";
    default:                 return "unknown";
    }
}

/* ------------------------------------------------------------------ */
/* Bounded Buffer                                                     */
/* ------------------------------------------------------------------ */

static int bounded_buffer_init(bounded_buffer_t *buf)
{
    int rc;
    memset(buf, 0, sizeof(*buf));

    rc = pthread_mutex_init(&buf->mutex, NULL);
    if (rc != 0) return rc;

    rc = pthread_cond_init(&buf->not_empty, NULL);
    if (rc != 0) {
        pthread_mutex_destroy(&buf->mutex);
        return rc;
    }
    rc = pthread_cond_init(&buf->not_full, NULL);
    if (rc != 0) {
        pthread_cond_destroy(&buf->not_empty);
        pthread_mutex_destroy(&buf->mutex);
        return rc;
    }
    return 0;
}
```

```c
static void bounded_buffer_destroy(bounded_buffer_t *buf)
{
    pthread_cond_destroy(&buf->not_full);
    pthread_cond_destroy(&buf->not_empty);
    pthread_mutex_destroy(&buf->mutex);
}

static void bounded_buffer_begin_shutdown(bounded_buffer_t *buf)
{
    pthread_mutex_lock(&buf->mutex);
    buf->shutting_down = 1;
    pthread_cond_broadcast(&buf->not_empty);
    pthread_cond_broadcast(&buf->not_full);
    pthread_mutex_unlock(&buf->mutex);
}

/*
 * bounded_buffer_push: producer side.
 *
 * Blocks when the buffer is full, until space opens or shutdown begins.
 * Returns 0 on success, -1 if we should stop (shutdown).
 *
 * Without the mutex + not_full condvar: two producers could both see
 * `count < capacity` simultaneously and write to the same slot →
 * data corruption. With mutex we prevent that race.
 *
 * Without not_empty broadcast after push: a waiting consumer would
 * never wake up → deadlock.
 */
int bounded_buffer_push(bounded_buffer_t *buf, const log_item_t *item)
{
    pthread_mutex_lock(&buf->mutex);

    while (buf->count == LOG_BUFFER_CAPACITY && !buf->shutting_down)
        pthread_cond_wait(&buf->not_full, &buf->mutex);

    if (buf->shutting_down) {
        pthread_mutex_unlock(&buf->mutex);
        return -1;
    }

    buf->items[buf->tail] = *item;
    buf->tail = (buf->tail + 1) % LOG_BUFFER_CAPACITY;
    buf->count++;

    pthread_cond_signal(&buf->not_empty);
    pthread_mutex_unlock(&buf->mutex);
    return 0;
}
```

```c
/*
 * bounded_buffer_pop: consumer side.
 *
 * Blocks when buffer is empty. On shutdown, drains remaining items
 * before returning -1 so no log lines are dropped.
 *
 * Returns 0 on success (item filled in), -1 when drained and shutdown.
 */
int bounded_buffer_pop(bounded_buffer_t *buf, log_item_t *item)
{
    pthread_mutex_lock(&buf->mutex);

    while (buf->count == 0) {
        if (buf->shutting_down) {
            pthread_mutex_unlock(&buf->mutex);
            return -1;
        }
        pthread_cond_wait(&buf->not_empty, &buf->mutex);
    }

    *item = buf->items[buf->head];
    buf->head = (buf->head + 1) % LOG_BUFFER_CAPACITY;
    buf->count--;

    pthread_cond_signal(&buf->not_full);
    pthread_mutex_unlock(&buf->mutex);
    return 0;
}

/* ---------------------------------------------------------------- */
/* Logging consumer thread                                          */
/* ---------------------------------------------------------------- */

void *logging_thread(void *arg)
{
    supervisor_ctx_t *ctx = (supervisor_ctx_t *)arg;
    log_item_t item;

    while (bounded_buffer_pop(&ctx->log_buffer, &item) == 0) {
        /* Find the container record to get log_path */
        char path[PATH_MAX];
        path[0] = '\0';

        pthread_mutex_lock(&ctx->metadata_lock);
        {
            container_record_t *c = ctx->containers;
            while (c) {
                if (strncmp(c->id, item.container_id, CONTAINER_ID_LEN) == 0) {
                    strncpy(path, c->log_path, PATH_MAX - 1);
```

```
                    path[PATH_MAX - 1] = '\0';
                    break;
                }
                c = c->next;
            }
        }
        pthread_mutex_unlock(&ctx->metadata_lock);

        if (path[0] == '\0') {
            /* Fallback: write to generic log file */
            snprintf(path, sizeof(path), "%s/%s.log",
                     LOG_DIR, item.container_id);
        }

        int fd = open(path, O_CREAT | O_WRONLY | O_APPEND, 0644);
        if (fd >= 0) {
            ssize_t n = write(fd, item.data, item.length);
            (void)n;
            close(fd);
        }
    }

    /* Drain remaining items after shutdown signal */
    {
        bounded_buffer_t *buf = &ctx->log_buffer;
        pthread_mutex_lock(&buf->mutex);
        while (buf->count > 0) {
            log_item_t leftover = buf->items[buf->head];
            buf->head = (buf->head + 1) % LOG_BUFFER_CAPACITY;
            buf->count--;
            pthread_mutex_unlock(&buf->mutex);

            char path[PATH_MAX];
            snprintf(path, sizeof(path), "%s/%s.log",
                     LOG_DIR, leftover.container_id);
            int fd = open(path, O_CREAT | O_WRONLY | O_APPEND, 0644);
            if (fd >= 0) {
                ssize_t n = write(fd, leftover.data, leftover.length);
                (void)n;
                close(fd);
            }

            pthread_mutex_lock(&buf->mutex);
        }
        pthread_mutex_unlock(&buf->mutex);
    }

    return NULL;
}
```

```c
/* ---------------------------------------------------------------- */
/* Producer thread — reads one container's pipe and pushes to buffer  */
/* ---------------------------------------------------------------- */

typedef struct {
    int              read_fd;
    char             container_id[CONTAINER_ID_LEN];
    bounded_buffer_t *buffer;
} producer_args_t;

static void *producer_thread(void *arg)
{
    producer_args_t *pa = (producer_args_t *)arg;
    char tmp[LOG_CHUNK_SIZE];
    ssize_t n;

    while ((n = read(pa->read_fd, tmp, sizeof(tmp) - 1)) > 0) {
        log_item_t item;
        memset(&item, 0, sizeof(item));
        snprintf(item.container_id, sizeof(item.container_id), "%s", pa->container_id);
        item.length = (size_t)n;
        memcpy(item.data, tmp, (size_t)n);
        bounded_buffer_push(pa->buffer, &item);
    }

    close(pa->read_fd);
    free(pa);
    return NULL;
}

/* ---------------------------------------------------------------- */
/* Container child entrypoint (runs after clone())                  */
/* ---------------------------------------------------------------- */

int child_fn(void *arg)
{
#ifdef __linux__
    child_config_t *cfg = (child_config_t *)arg;

    /* Redirect stdout and stderr to the logging pipe */
    if (dup2(cfg->log_write_fd, STDOUT_FILENO) < 0 ||
        dup2(cfg->log_write_fd, STDERR_FILENO) < 0) {
        perror("dup2");
        return 1;
    }
    close(cfg->log_write_fd);

    /* Set nice value */
    if (cfg->nice_value != 0) {
        if (nice(cfg->nice_value) == -1 && errno != 0)
```

```
            perror("nice");   /* non-fatal */
    }

    /* Set hostname (UTS namespace) */
    if (sethostname(cfg->id, strlen(cfg->id)) < 0)
        perror("sethostname");   /* non-fatal */

    /* chroot into container rootfs */
    if (chroot(cfg->rootfs) < 0) {
        perror("chroot");
        return 1;
    }
    if (chdir("/") < 0) {
        perror("chdir");
        return 1;
    }

    /* Mount /proc so 'ps' and other tools work inside the container.
     * Linux mount(2) takes 5 arguments: source, target, fstype, flags, data.
     */
    mkdir("/proc", 0555);
    if (mount("proc", "/proc", "proc", 0, NULL) < 0)
        perror("mount /proc");   /* non-fatal if already mounted */

    /* Execute the requested command */
    char *argv_exec[] = { cfg->command, NULL };
    execv(cfg->command, argv_exec);

    /* If execv returns, try execvp with PATH lookup */
    char *argv_p[] = { cfg->command, NULL };
    execvp(cfg->command, argv_p);

    perror("exec");
    return 127;
#else
    (void)arg;
    fprintf(stderr, "child_fn: Linux-only (clone/namespaces not available)\n");
    return 1;
#endif
}

/* ------------------------------------------------------------------ */
/* Monitor device helpers                                              */
/* ------------------------------------------------------------------ */

int register_with_monitor(int monitor_fd,
                          const char *container_id,
                          pid_t host_pid,
                          unsigned long soft_limit_bytes,
                          unsigned long hard_limit_bytes)
```

```c
{
    struct monitor_request req;
    memset(&req, 0, sizeof(req));
    req.pid              = host_pid;
    req.soft_limit_bytes = soft_limit_bytes;
    req.hard_limit_bytes = hard_limit_bytes;
    snprintf(req.container_id, sizeof(req.container_id), "%s", container_id);

    if (ioctl(monitor_fd, MONITOR_REGISTER, &req) < 0)
        return -1;
    return 0;
}

int unregister_from_monitor(int monitor_fd,
                            const char *container_id,
                            pid_t host_pid)
{
    struct monitor_request req;
    memset(&req, 0, sizeof(req));
    req.pid = host_pid;
    snprintf(req.container_id, sizeof(req.container_id), "%s", container_id);

    if (ioctl(monitor_fd, MONITOR_UNREGISTER, &req) < 0)
        return -1;
    return 0;
}

/* ---------------------------------------------------------------- */
/* Container metadata helpers                                       */
/* ---------------------------------------------------------------- */

static container_record_t *find_container(supervisor_ctx_t *ctx,
                                          const char *id)
{
    container_record_t *c = ctx->containers;
    while (c) {
        if (strncmp(c->id, id, CONTAINER_ID_LEN) == 0)
            return c;
        c = c->next;
    }
    return NULL;
}

static void add_container(supervisor_ctx_t *ctx, container_record_t *rec)
{
    rec->next = ctx->containers;
    ctx->containers = rec;
}

/* ---------------------------------------------------------------- */
```

```c
/* SIGCHLD handler and signal infrastructure                          */
/* ------------------------------------------------------------------ */

static volatile sig_atomic_t g_sigchld_pending = 0;
static volatile sig_atomic_t g_shutdown        = 0;

static void sigchld_handler(int sig)
{
    (void)sig;
    g_sigchld_pending = 1;
}

static void sigterm_handler(int sig)
{
    (void)sig;
    g_shutdown = 1;
}

static void reap_children(supervisor_ctx_t *ctx)
{
    int status;
    pid_t pid;

    while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
        pthread_mutex_lock(&ctx->metadata_lock);
        container_record_t *c = ctx->containers;
        while (c) {
            if (c->host_pid == pid) {
                if (WIFEXITED(status)) {
                    c->exit_code   = WEXITSTATUS(status);
                    c->exit_signal = 0;
                    c->state       = c->stop_requested
                                     ? CONTAINER_STOPPED
                                     : CONTAINER_EXITED;
                } else if (WIFSIGNALED(status)) {
                    c->exit_signal = WTERMSIG(status);
                    c->exit_code   = 128 + c->exit_signal;
                    if (c->stop_requested)
                        c->state = CONTAINER_STOPPED;
                    else if (c->exit_signal == SIGKILL)
                        c->state = CONTAINER_KILLED;
                    else
                        c->state = CONTAINER_EXITED;
                }
                /* Unregister from kernel monitor */
                if (ctx->monitor_fd >= 0)
                    unregister_from_monitor(ctx->monitor_fd, c->id, pid);
                break;
            }
            c = c->next;
```

```c
        }
        pthread_mutex_unlock(&ctx->metadata_lock);
    }
}

/* ---------------------------------------------------------------- */
/* Launch a container (used by both start and run commands)          */
/* ---------------------------------------------------------------- */

static int launch_container(supervisor_ctx_t *ctx,
                            const control_request_t *req,
                            control_response_t *resp)
{
    int pipefd[2];
    container_record_t *rec;
    child_config_t *cfg;
    char *stack;
    pid_t pid;

    /* Check ID uniqueness */
    pthread_mutex_lock(&ctx->metadata_lock);
    if (find_container(ctx, req->container_id)) {
        pthread_mutex_unlock(&ctx->metadata_lock);
        snprintf(resp->message, sizeof(resp->message),
                 "container '%s' already exists", req->container_id);
        resp->status = -1;
        return -1;
    }
    pthread_mutex_unlock(&ctx->metadata_lock);

    /* Create the logging pipe */
    if (pipe(pipefd) < 0) {
        perror("pipe");
        snprintf(resp->message, sizeof(resp->message), "pipe: %s", strerror(errno));
        resp->status = -1;
        return -1;
    }

    /* Ensure log directory exists */
    mkdir(LOG_DIR, 0755);

    /* Allocate container record */
    rec = calloc(1, sizeof(*rec));
    if (!rec) {
        close(pipefd[0]); close(pipefd[1]);
        resp->status = -1;
        snprintf(resp->message, sizeof(resp->message), "OOM");
        return -1;
    }
    snprintf(rec->id, sizeof(rec->id), "%s", req->container_id);
```

```c
    rec->started_at       = time(NULL);
    rec->state            = CONTAINER_STARTING;
    rec->soft_limit_bytes = req->soft_limit_bytes;
    rec->hard_limit_bytes = req->hard_limit_bytes;
    snprintf(rec->log_path, PATH_MAX, "%s/%s.log", LOG_DIR, req->container_id);

    /* Allocate child config */
    cfg = calloc(1, sizeof(*cfg));
    if (!cfg) {
        free(rec);
        close(pipefd[0]); close(pipefd[1]);
        resp->status = -1;
        snprintf(resp->message, sizeof(resp->message), "OOM");
        return -1;
    }
    snprintf(cfg->id, sizeof(cfg->id), "%s", req->container_id);
    snprintf(cfg->rootfs, sizeof(cfg->rootfs), "%s", req->rootfs);
    snprintf(cfg->command, sizeof(cfg->command), "%s", req->command);
    cfg->nice_value   = req->nice_value;
    cfg->log_write_fd = pipefd[1];

    /* Allocate clone stack */
    stack = malloc(STACK_SIZE);
    if (!stack) {
        free(cfg); free(rec);
        close(pipefd[0]); close(pipefd[1]);
        resp->status = -1;
        snprintf(resp->message, sizeof(resp->message), "OOM");
        return -1;
    }

    /* Clone with isolated namespaces (Linux-only) */
#ifdef __linux__
    pid = clone(child_fn, stack + STACK_SIZE,
                CLONE_NEWPID | CLONE_NEWUTS | CLONE_NEWNS | SIGCHLD,
                cfg);
    if (pid < 0) {
        perror("clone");
        free(stack); free(cfg); free(rec);
        close(pipefd[0]); close(pipefd[1]);
        snprintf(resp->message, sizeof(resp->message),
                 "clone failed: %s", strerror(errno));
        resp->status = -1;
        return -1;
    }
#else
    (void)child_fn;
    free(stack); free(cfg); free(rec);
    close(pipefd[0]); close(pipefd[1]);
    snprintf(resp->message, sizeof(resp->message),
```

```
            "clone/namespaces not available on this platform");
    resp->status = -1;
    return -1;
    pid = -1; /* unreachable, suppress compiler warning */
#endif

    /* Parent: close write end; start producer thread on read end */
    close(pipefd[1]);

    producer_args_t *pa = malloc(sizeof(*pa));
    if (pa) {
        pa->read_fd = pipefd[0];
        strncpy(pa->container_id, req->container_id, CONTAINER_ID_LEN - 1);
        pa->container_id[CONTAINER_ID_LEN - 1] = '\0';
        pa->buffer = &ctx->log_buffer;
        pthread_t prod_tid;
        pthread_attr_t attr;
        pthread_attr_init(&attr);
        pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
        pthread_create(&prod_tid, &attr, producer_thread, pa);
        pthread_attr_destroy(&attr);
    } else {
        close(pipefd[0]);
    }

    /* Register with kernel monitor */
    if (ctx->monitor_fd >= 0) {
        register_with_monitor(ctx->monitor_fd, req->container_id, pid,
                              req->soft_limit_bytes, req->hard_limit_bytes);
    }

    /* Update metadata */
    rec->host_pid = pid;
    rec->state    = CONTAINER_RUNNING;
    free(stack);
    free(cfg);

    pthread_mutex_lock(&ctx->metadata_lock);
    add_container(ctx, rec);
    pthread_mutex_unlock(&ctx->metadata_lock);

    snprintf(resp->message, sizeof(resp->message),
             "started container '%s' pid=%d", req->container_id, pid);
    resp->status = 0;
    return pid;
}

/* ------------------------------------------------------------------- */
/* Handle one control request from a connected CLI client              */
/* ------------------------------------------------------------------- */
```

```c
static void handle_request(supervisor_ctx_t *ctx, int conn_fd)
{
    control_request_t  req;
    control_response_t resp;

    memset(&resp, 0, sizeof(resp));

    ssize_t n = recv(conn_fd, &req, sizeof(req), 0);
    if (n != (ssize_t)sizeof(req)) {
        resp.status = -1;
        snprintf(resp.message, sizeof(resp.message), "protocol error");
        send(conn_fd, &resp, sizeof(resp), 0);
        return;
    }

    switch (req.kind) {

    case CMD_START: {
        launch_container(ctx, &req, &resp);
        break;
    }

    case CMD_RUN: {
        int pid = launch_container(ctx, &req, &resp);
        if (pid > 0) {
            /* Block until container exits, then report exit status */
            int wstatus;
            waitpid(pid, &wstatus, 0);
            int exit_code = WIFEXITED(wstatus) ? WEXITSTATUS(wstatus)
                                               : 128 + WTERMSIG(wstatus);
            snprintf(resp.message, sizeof(resp.message),
                     "container '%s' exited with status %d",
                     req.container_id, exit_code);
            resp.status = exit_code;
        }
        break;
    }

    case CMD_PS: {
        char buf[CONTROL_MESSAGE_LEN * 8];
        int  off = 0;

        off += snprintf(buf + off, sizeof(buf) - off,
                        "%-16s %-8s %-10s %-12s %-12s\n",
                        "ID", "PID", "STATE", "SOFT-MiB", "HARD-MiB");

        pthread_mutex_lock(&ctx->metadata_lock);
        container_record_t *c = ctx->containers;
        while (c && off < (int)sizeof(buf) - 80) {
```

```
        off += snprintf(buf + off, sizeof(buf) - off,
                        "%-16s %-8d %-10s %-12lu %-12lu\n",
                        c->id, c->host_pid,
                        state_to_string(c->state),
                        c->soft_limit_bytes >> 20,
                        c->hard_limit_bytes >> 20);
        c = c->next;
    }
    pthread_mutex_unlock(&ctx->metadata_lock);

    resp.status = 0;
    snprintf(resp.message, sizeof(resp.message), "%s", buf);
    break;
}

case CMD_LOGS: {
    char log_path[PATH_MAX];
    log_path[0] = '\0';

    pthread_mutex_lock(&ctx->metadata_lock);
    container_record_t *c = find_container(ctx, req.container_id);
    if (c)
        snprintf(log_path, sizeof(log_path), "%s", c->log_path);
    pthread_mutex_unlock(&ctx->metadata_lock);

    if (log_path[0] == '\0') {
        resp.status = -1;
        snprintf(resp.message, sizeof(resp.message),
                 "container '%s' not found", req.container_id);
    } else {
        /* Read and send log file contents */
        int fd = open(log_path, O_RDONLY);
        if (fd < 0) {
            resp.status = -1;
            snprintf(resp.message, sizeof(resp.message),
                     "cannot open log: %s", strerror(errno));
        } else {
            resp.status = 0;
            snprintf(resp.message, sizeof(resp.message),
                     "log: %s", log_path);
            send(conn_fd, &resp, sizeof(resp), 0);

            /* Stream log content */
            char chunk[4096];
            ssize_t r;
            while ((r = read(fd, chunk, sizeof(chunk))) > 0)
                send(conn_fd, chunk, (size_t)r, 0);
            close(fd);
            return;
        }
```

```c
        }
        break;
    }

    case CMD_STOP: {
        pthread_mutex_lock(&ctx->metadata_lock);
        container_record_t *c = find_container(ctx, req.container_id);
        if (!c) {
            pthread_mutex_unlock(&ctx->metadata_lock);
            resp.status = -1;
            snprintf(resp.message, sizeof(resp.message),
                     "container '%s' not found", req.container_id);
        } else if (c->state != CONTAINER_RUNNING) {
            pid_t pid = c->host_pid;
            pthread_mutex_unlock(&ctx->metadata_lock);
            resp.status = -1;
            snprintf(resp.message, sizeof(resp.message),
                     "container '%s' (pid=%d) is not running", req.container_id, pid);
        } else {
            c->stop_requested = 1;
            pid_t pid = c->host_pid;
            pthread_mutex_unlock(&ctx->metadata_lock);

            kill(pid, SIGTERM);
            resp.status = 0;
            snprintf(resp.message, sizeof(resp.message),
                     "sent SIGTERM to container '%s' pid=%d",
                     req.container_id, pid);
        }
        break;
    }

    default:
        resp.status = -1;
        snprintf(resp.message, sizeof(resp.message), "unknown command %d", req.kind);
        break;
    }

    send(conn_fd, &resp, sizeof(resp), 0);
}

/* ------------------------------------------------------------------ */
/* Supervisor main loop                                               */
/* ------------------------------------------------------------------ */

static int run_supervisor(const char *rootfs)
{
    supervisor_ctx_t ctx;
    struct sockaddr_un addr;
    int rc;
```

```c
    memset(&ctx, 0, sizeof(ctx));
    ctx.server_fd  = -1;
    ctx.monitor_fd = -1;
    g_ctx = &ctx;

    (void)rootfs;   /* base rootfs available for future use */

    rc = pthread_mutex_init(&ctx.metadata_lock, NULL);
    if (rc != 0) {
        errno = rc;
        perror("pthread_mutex_init");
        return 1;
    }

    rc = bounded_buffer_init(&ctx.log_buffer);
    if (rc != 0) {
        errno = rc;
        perror("bounded_buffer_init");
        pthread_mutex_destroy(&ctx.metadata_lock);
        return 1;
    }

    /* Open kernel monitor (non-fatal if module not loaded) */
    ctx.monitor_fd = open("/dev/container_monitor", O_RDWR);
    if (ctx.monitor_fd < 0)
        fprintf(stderr, "[supervisor] warning: /dev/container_monitor not available: %s\n",
                strerror(errno));

    /* Create log directory */
    mkdir(LOG_DIR, 0755);

    /* Create UNIX domain socket */
    unlink(CONTROL_PATH);
    ctx.server_fd = socket(AF_UNIX, SOCK_STREAM, 0);
    if (ctx.server_fd < 0) {
        perror("socket");
        goto cleanup;
    }
    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strncpy(addr.sun_path, CONTROL_PATH, sizeof(addr.sun_path) - 1);

    if (bind(ctx.server_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("bind");
        goto cleanup;
    }
    if (listen(ctx.server_fd, 8) < 0) {
        perror("listen");
        goto cleanup;
```

```
    }

    /* Install signal handlers */
    {
        struct sigaction sa;
        memset(&sa, 0, sizeof(sa));
        sa.sa_handler = sigchld_handler;
        sigemptyset(&sa.sa_mask);
        sa.sa_flags = SA_RESTART | SA_NOCLDSTOP;
        sigaction(SIGCHLD, &sa, NULL);

        sa.sa_handler = sigterm_handler;
        sigaction(SIGTERM, &sa, NULL);
        sigaction(SIGINT,  &sa, NULL);
    }

    /* Start logger consumer thread */
    rc = pthread_create(&ctx.logger_thread, NULL, logging_thread, &ctx);
    if (rc != 0) {
        errno = rc;
        perror("pthread_create logger");
        goto cleanup;
    }

    fprintf(stderr, "[supervisor] started, control socket: %s\n", CONTROL_PATH);

    /* Event loop */
    while (!g_shutdown) {
        /* Reap any pending children */
        if (g_sigchld_pending) {
            g_sigchld_pending = 0;
            reap_children(&ctx);
        }

        fd_set rfds;
        FD_ZERO(&rfds);
        FD_SET(ctx.server_fd, &rfds);
        struct timeval tv = {0, 200000};  /* 200ms timeout to check g_shutdown */

        int sel = select(ctx.server_fd + 1, &rfds, NULL, NULL, &tv);
        if (sel < 0) {
            if (errno == EINTR) continue;
            perror("select");
            break;
        }
        if (sel == 0) continue;

        int conn_fd = accept(ctx.server_fd, NULL, NULL);
        if (conn_fd < 0) {
            if (errno == EINTR) continue;
```

```
            perror("accept");
            continue;
        }

        handle_request(&ctx, conn_fd);
        close(conn_fd);
    }

    fprintf(stderr, "[supervisor] shutting down...\n");

    /* Kill all running containers */
    pthread_mutex_lock(&ctx.metadata_lock);
    {
        container_record_t *c = ctx.containers;
        while (c) {
            if (c->state == CONTAINER_RUNNING) {
                c->stop_requested = 1;
                kill(c->host_pid, SIGTERM);
            }
            c = c->next;
        }
    }
    pthread_mutex_unlock(&ctx.metadata_lock);

    /* Brief wait for containers to exit */
    {
        int wstatus;
        while (waitpid(-1, &wstatus, WNOHANG) > 0)
            ;
        usleep(500000);
        while (waitpid(-1, &wstatus, WNOHANG) > 0)
            ;
    }

cleanup:
    bounded_buffer_begin_shutdown(&ctx.log_buffer);
    if (ctx.logger_thread)
        pthread_join(ctx.logger_thread, NULL);

    bounded_buffer_destroy(&ctx.log_buffer);

    /* Free container list */
    pthread_mutex_lock(&ctx.metadata_lock);
    {
        container_record_t *c = ctx.containers;
        while (c) {
            container_record_t *next = c->next;
            free(c);
            c = next;
        }
```

```
        ctx.containers = NULL;
    }
    pthread_mutex_unlock(&ctx.metadata_lock);
    pthread_mutex_destroy(&ctx.metadata_lock);

    if (ctx.server_fd >= 0) close(ctx.server_fd);
    if (ctx.monitor_fd >= 0) close(ctx.monitor_fd);
    unlink(CONTROL_PATH);

    fprintf(stderr, "[supervisor] exited cleanly.\n");
    return 0;
}

/* ------------------------------------------------------------------- */
/* Client side: send a control request to the supervisor               */
/* ------------------------------------------------------------------- */

static int send_control_request(const control_request_t *req)
{
    int sock;
    struct sockaddr_un addr;
    control_response_t resp;

    sock = socket(AF_UNIX, SOCK_STREAM, 0);
    if (sock < 0) {
        perror("socket");
        return 1;
    }

    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strncpy(addr.sun_path, CONTROL_PATH, sizeof(addr.sun_path) - 1);

    if (connect(sock, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        fprintf(stderr, "Cannot connect to supervisor at %s: %s\n"
                "Is the supervisor running?\n",
                CONTROL_PATH, strerror(errno));
        close(sock);
        return 1;
    }

    if (send(sock, req, sizeof(*req), 0) != (ssize_t)sizeof(*req)) {
        perror("send");
        close(sock);
        return 1;
    }

    ssize_t n = recv(sock, &resp, sizeof(resp), 0);
    if (n <= 0) {
        perror("recv");
```

```
        close(sock);
        return 1;
    }

    /* For LOG commands, stream additional data */
    if (req->kind == CMD_LOGS && resp.status == 0) {
        printf("%s\n", resp.message);
        char chunk[4096];
        ssize_t r;
        while ((r = recv(sock, chunk, sizeof(chunk), 0)) > 0) {
            fwrite(chunk, 1, (size_t)r, stdout);
        }
    } else {
        printf("%s\n", resp.message);
    }

    close(sock);
    return (resp.status == 0) ? 0 : 1;
}

/* ------------------------------------------------------------------ */
/* CLI command handlers (client side)                                 */
/* ------------------------------------------------------------------ */

static int cmd_start(int argc, char *argv[])
{
    control_request_t req;

    if (argc < 5) {
        fprintf(stderr,
                "Usage: %s start <id> <container-rootfs> <command> [--soft-mib N] [--hard-
mib N] [--nice N]\n",
                argv[0]);
        return 1;
    }

    memset(&req, 0, sizeof(req));
    req.kind = CMD_START;
    strncpy(req.container_id, argv[2], sizeof(req.container_id) - 1);
    strncpy(req.rootfs, argv[3], sizeof(req.rootfs) - 1);
    strncpy(req.command, argv[4], sizeof(req.command) - 1);
    req.soft_limit_bytes = DEFAULT_SOFT_LIMIT;
    req.hard_limit_bytes = DEFAULT_HARD_LIMIT;

    if (parse_optional_flags(&req, argc, argv, 5) != 0)
        return 1;

    return send_control_request(&req);
}
```

```c
static int cmd_run(int argc, char *argv[])
{
    control_request_t req;

    if (argc < 5) {
        fprintf(stderr,
                "Usage: %s run <id> <container-rootfs> <command> [--soft-mib N] [--hard-mib
N] [--nice N]\n",
                argv[0]);
        return 1;
    }

    memset(&req, 0, sizeof(req));
    req.kind = CMD_RUN;
    strncpy(req.container_id, argv[2], sizeof(req.container_id) - 1);
    strncpy(req.rootfs, argv[3], sizeof(req.rootfs) - 1);
    strncpy(req.command, argv[4], sizeof(req.command) - 1);
    req.soft_limit_bytes = DEFAULT_SOFT_LIMIT;
    req.hard_limit_bytes = DEFAULT_HARD_LIMIT;

    if (parse_optional_flags(&req, argc, argv, 5) != 0)
        return 1;

    return send_control_request(&req);
}

static int cmd_ps(void)
{
    control_request_t req;
    memset(&req, 0, sizeof(req));
    req.kind = CMD_PS;
    return send_control_request(&req);
}

static int cmd_logs(int argc, char *argv[])
{
    control_request_t req;

    if (argc < 3) {
        fprintf(stderr, "Usage: %s logs <id>\n", argv[0]);
        return 1;
    }

    memset(&req, 0, sizeof(req));
    req.kind = CMD_LOGS;
    strncpy(req.container_id, argv[2], sizeof(req.container_id) - 1);
    return send_control_request(&req);
}

static int cmd_stop(int argc, char *argv[])
```

```c
{
    control_request_t req;

    if (argc < 3) {
        fprintf(stderr, "Usage: %s stop <id>\n", argv[0]);
        return 1;
    }

    memset(&req, 0, sizeof(req));
    req.kind = CMD_STOP;
    strncpy(req.container_id, argv[2], sizeof(req.container_id) - 1);
    return send_control_request(&req);
}

/* ------------------------------------------------------------------ */
/* main                                                               */
/* ------------------------------------------------------------------ */

int main(int argc, char *argv[])
{
    if (argc < 2) {
        usage(argv[0]);
        return 1;
    }

    if (strcmp(argv[1], "supervisor") == 0) {
        if (argc < 3) {
            fprintf(stderr, "Usage: %s supervisor <base-rootfs>\n", argv[0]);
            return 1;
        }
        return run_supervisor(argv[2]);
    }

    if (strcmp(argv[1], "start") == 0) return cmd_start(argc, argv);
    if (strcmp(argv[1], "run")   == 0) return cmd_run(argc, argv);
    if (strcmp(argv[1], "ps")    == 0) return cmd_ps();
    if (strcmp(argv[1], "logs")  == 0) return cmd_logs(argc, argv);
    if (strcmp(argv[1], "stop")  == 0) return cmd_stop(argc, argv);

    usage(argv[0]);
    return 1;
}
```

4.3 Monitor.c

```c
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/fs.h>
#include <linux/kernel.h>
#include <linux/list.h>
#include <linux/mm.h>
#include <linux/module.h>
#include <linux/spinlock.h>
#include <linux/pid.h>
#include <linux/sched/signal.h>
#include <linux/slab.h>
#include <linux/timer.h>
#include <linux/uaccess.h>
#include <linux/version.h>

#include "monitor_ioctl.h"

#define DEVICE_NAME        "container_monitor"
#define CHECK_INTERVAL_SEC 1

/* ---------------------------------------------------------------
 * Data structure: one node per tracked container.
 * --------------------------------------------------------------- */
struct monitored_entry {
    pid_t          pid;
    char           container_id[MONITOR_NAME_LEN];
    unsigned long  soft_limit_bytes;
    unsigned long  hard_limit_bytes;
    int            soft_warned;   /* 1 after first soft warning */
    struct list_head node;
};

/* ---------------------------------------------------------------
 * Global state: shared list + a spinlock to protect it.
 * Spinlock is mandatory because the timer callback fires in softirq
 * context (cannot sleep). spin_lock_irqsave is used everywhere so
 * the same lock is safe from process context as well.
 * --------------------------------------------------------------- */
static LIST_HEAD(monitored_list);
static DEFINE_SPINLOCK(monitored_lock);

/* --- Provided: internal device / timer state --- */
static struct timer_list monitor_timer;
static dev_t             dev_num;
static struct cdev       c_dev;
static struct class     *cl;

/* ---------------------------------------------------------------
```

```
 * Provided: RSS Helper
 * Returns the Resident Set Size in bytes for the given PID,
 * or -1 if the task no longer exists.
 * ---------------------------------------------------------------- */
static long get_rss_bytes(pid_t pid)
{
    struct task_struct *task;
    struct mm_struct   *mm;
    long               rss_pages = 0;

    rcu_read_lock();
    task = pid_task(find_vpid(pid), PIDTYPE_PID);
    if (!task) {
        rcu_read_unlock();
        return -1;
    }
    get_task_struct(task);
    rcu_read_unlock();

    mm = get_task_mm(task);
    if (mm) {
        rss_pages = get_mm_rss(mm);
        mmput(mm);
    }
    put_task_struct(task);

    return rss_pages * PAGE_SIZE;
}

/* ----------------------------------------------------------------
 * Provided: soft-limit warning helper
 * ---------------------------------------------------------------- */
static void log_soft_limit_event(const char *container_id,
                                 pid_t        pid,
                                 unsigned long limit_bytes,
                                 long          rss_bytes)
{
    printk(KERN_WARNING
           "[container_monitor] SOFT LIMIT container=%s pid=%d rss=%ld limit=%lu\n",
           container_id, pid, rss_bytes, limit_bytes);
}

/* ----------------------------------------------------------------
 * Provided: hard-limit enforcement helper
 * ---------------------------------------------------------------- */
static void kill_process(const char   *container_id,
                         pid_t         pid,
                         unsigned long limit_bytes,
                         long          rss_bytes)
{
```

```
    struct task_struct *task;

    rcu_read_lock();
    task = pid_task(find_vpid(pid), PIDTYPE_PID);
    if (task)
        send_sig(SIGKILL, task, 1);
    rcu_read_unlock();

    printk(KERN_WARNING
           "[container_monitor] HARD LIMIT container=%s pid=%d rss=%ld limit=%lu\n",
           container_id, pid, rss_bytes, limit_bytes);
}

/* ----------------------------------------------------------------
 * Timer Callback - fires every CHECK_INTERVAL_SEC seconds.
 *
 * For every monitored entry:
 *   1. Call get_rss_bytes().
 *   2. If the task no longer exists, remove the stale entry.
 *   3. If RSS > hard limit, kill and remove the entry.
 *   4. If RSS > soft limit and we haven't warned yet, warn once.
 * ---------------------------------------------------------------- */
static void timer_callback(struct timer_list *t)
{
    struct monitored_entry *entry, *tmp;
    unsigned long flags;

    spin_lock_irqsave(&monitored_lock, flags);

    list_for_each_entry_safe(entry, tmp, &monitored_list, node) {
        long rss = get_rss_bytes(entry->pid);

        /* Task no longer exists — remove stale entry */
        if (rss < 0) {
            printk(KERN_INFO
                   "[container_monitor] container=%s pid=%d exited, removing\n",
                   entry->container_id, entry->pid);
            list_del(&entry->node);
            kfree(entry);
            continue;
        }

        /* Hard limit: kill and remove */
        if ((unsigned long)rss > entry->hard_limit_bytes) {
            kill_process(entry->container_id, entry->pid,
                         entry->hard_limit_bytes, rss);
            list_del(&entry->node);
            kfree(entry);
            continue;
        }
```

```
        /* Soft limit: warn once */
        if ((unsigned long)rss > entry->soft_limit_bytes && !entry->soft_warned) {
            log_soft_limit_event(entry->container_id, entry->pid,
                                 entry->soft_limit_bytes, rss);
            entry->soft_warned = 1;
        }
    }

    spin_unlock_irqrestore(&monitored_lock, flags);

    mod_timer(&monitor_timer, jiffies + CHECK_INTERVAL_SEC * HZ);
}

/* ---------------------------------------------------------------
 * IOCTL Handler
 * --------------------------------------------------------------- */
static long monitor_ioctl(struct file *f, unsigned int cmd, unsigned long arg)
{
    struct monitor_request req;

    (void)f;

    if (cmd != MONITOR_REGISTER && cmd != MONITOR_UNREGISTER)
        return -EINVAL;

    if (copy_from_user(&req, (struct monitor_request __user *)arg, sizeof(req)))
        return -EFAULT;

    /* Null-terminate container_id defensively */
    req.container_id[MONITOR_NAME_LEN - 1] = '\0';

    if (cmd == MONITOR_REGISTER) {
        struct monitored_entry *entry;

        printk(KERN_INFO
               "[container_monitor] Registering container=%s pid=%d soft=%lu hard=%lu\n",
               req.container_id, req.pid,
               req.soft_limit_bytes, req.hard_limit_bytes);

        if (req.soft_limit_bytes > req.hard_limit_bytes) {
            printk(KERN_WARNING
                   "[container_monitor] Invalid limits for container=%s\n",
                   req.container_id);
            return -EINVAL;
        }

        entry = kmalloc(sizeof(*entry), GFP_KERNEL);
        if (!entry)
            return -ENOMEM;
```

```
        entry->pid              = req.pid;
        entry->soft_limit_bytes = req.soft_limit_bytes;
        entry->hard_limit_bytes = req.hard_limit_bytes;
        entry->soft_warned      = 0;
        strncpy(entry->container_id, req.container_id, MONITOR_NAME_LEN - 1);
        entry->container_id[MONITOR_NAME_LEN - 1] = '\0';
        INIT_LIST_HEAD(&entry->node);

        {
            unsigned long flags;
            spin_lock_irqsave(&monitored_lock, flags);
            list_add_tail(&entry->node, &monitored_list);
            spin_unlock_irqrestore(&monitored_lock, flags);
        }

        return 0;
    }

    /* MONITOR_UNREGISTER */
    printk(KERN_INFO
           "[container_monitor] Unregister request container=%s pid=%d\n",
           req.container_id, req.pid);

    {
        struct monitored_entry *entry, *tmp;
        int found = 0;

        {
            unsigned long flags;
            spin_lock_irqsave(&monitored_lock, flags);
            list_for_each_entry_safe(entry, tmp, &monitored_list, node) {
                if (entry->pid == req.pid &&
                    strncmp(entry->container_id, req.container_id,
                            MONITOR_NAME_LEN) == 0) {
                    list_del(&entry->node);
                    kfree(entry);
                    found = 1;
                    break;
                }
            }
            spin_unlock_irqrestore(&monitored_lock, flags);
        }

        return found ? 0 : -ENOENT;
    }
}

/* --- Provided: file operations --- */
static struct file_operations fops = {
```

```
    .owner          = THIS_MODULE,
    .unlocked_ioctl = monitor_ioctl,
};

/* --- Module Init --- */
static int __init monitor_init(void)
{
    if (alloc_chrdev_region(&dev_num, 0, 1, DEVICE_NAME) < 0)
        return -1;

#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 4, 0)
    cl = class_create(DEVICE_NAME);
#else
    cl = class_create(THIS_MODULE, DEVICE_NAME);
#endif
    if (IS_ERR(cl)) {
        unregister_chrdev_region(dev_num, 1);
        return PTR_ERR(cl);
    }

    if (IS_ERR(device_create(cl, NULL, dev_num, NULL, DEVICE_NAME))) {
        class_destroy(cl);
        unregister_chrdev_region(dev_num, 1);
        return -1;
    }

    cdev_init(&c_dev, &fops);
    if (cdev_add(&c_dev, dev_num, 1) < 0) {
        device_destroy(cl, dev_num);
        class_destroy(cl);
        unregister_chrdev_region(dev_num, 1);
        return -1;
    }

    timer_setup(&monitor_timer, timer_callback, 0);
    mod_timer(&monitor_timer, jiffies + CHECK_INTERVAL_SEC * HZ);

    printk(KERN_INFO "[container_monitor] Module loaded. Device: /dev/%s\n", DEVICE_NAME);
    return 0;
}

/* --- Module Exit --- */
static void __exit monitor_exit(void)
{
    struct monitored_entry *entry, *tmp;
    unsigned long flags;

    /* del_timer_sync was renamed timer_delete_sync in Linux 6.15 */
#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 15, 0)
    timer_delete_sync(&monitor_timer);
```

```
#else
    del_timer_sync(&monitor_timer);
#endif

    /* Free all remaining monitored entries */
    spin_lock_irqsave(&monitored_lock, flags);
    list_for_each_entry_safe(entry, tmp, &monitored_list, node) {
        list_del(&entry->node);
        kfree(entry);
    }
    spin_unlock_irqrestore(&monitored_lock, flags);

    cdev_del(&c_dev);
    device_destroy(cl, dev_num);
    class_destroy(cl);
    unregister_chrdev_region(dev_num, 1);

    printk(KERN_INFO "[container_monitor] Module unloaded.\n");
}

module_init(monitor_init);
module_exit(monitor_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Supervised multi-container memory monitor");
```

CHAPTER 5
RESULT AND DISCUSSION

5.1 Multi-Container Runtime Results

The supervisor successfully launches and tracks multiple containers simultaneously. Each container receives isolated PID, UTS, and mount namespaces. Processes inside one container cannot observe the process tree of another container or of the host. The ps command correctly displays all tracked containers with their host PIDs, start times, states, and memory limit configuration. SIGCHLD is handled correctly and no zombie processes are observed after containers exit, confirming reliable child reaping.

5.2 Logging Pipeline Results

The bounded-buffer logging pipeline captures all container output without data loss. Producer threads correctly block on pthread_cond_wait when the buffer is full, and the consumer thread drains the buffer at a sufficient rate for normal workload output. Per-container log files persist after the container exits and contain the complete output of the container session. No deadlocks were observed during testing with up to four concurrent containers. The choice of pthread_mutex with condition variables over semaphores provides flexibility for future extension to a multi-consumer design.

5.3 Kernel Memory Enforcement Results

The kernel module successfully registers container PIDs via ioctl and periodically monitors their RSS. When a test process allocates memory incrementally beyond the configured soft limit, a warning message appears in dmesg identifying the PID and its current RSS. When the same process crosses the hard limit, SIGKILL is delivered, the process terminates, and the supervisor updates the container metadata to reflect the killed state and reason. The kernel linked list is correctly cleaned on module unload with no memory leaks or kernel warnings detected.

5.4 Scheduler Experiment Results

Experiment 1 ran two CPU-bound containers at nice 0. Both received approximately equal CPU shares, consistent with CFS fairness goals. Experiment 2 ran one CPU-bound container at nice 0 and another at

nice +10. The lower-priority container completed the same fixed computation in approximately 2.8x the time of the higher-priority container, demonstrating that CFS correctly assigns less CPU time to tasks with higher nice values. Experiment 3 ran a CPU-bound container alongside an I/O-bound container at equal priorities. The I/O-bound container remained responsive because it spent most of its time blocked on I/O, accumulating less vruntime and therefore receiving prompt scheduling when it woke. These results confirm that CFS achieves fairness among competing CPU consumers while naturally favouring I/O-bound tasks in terms of responsiveness.

CHAPTER 6
OUTPUTS

The following section describes the expected output screenshots for each required demonstration scenario. Actual screenshots are to be inserted upon project completion.

Figure 6.1 - Multi-Container Supervision

Screenshot showing two containers (alpha and beta) running simultaneously under one supervisor process. The terminal displays the supervisor startup message and both container launch acknowledgements, including assigned host PIDs.

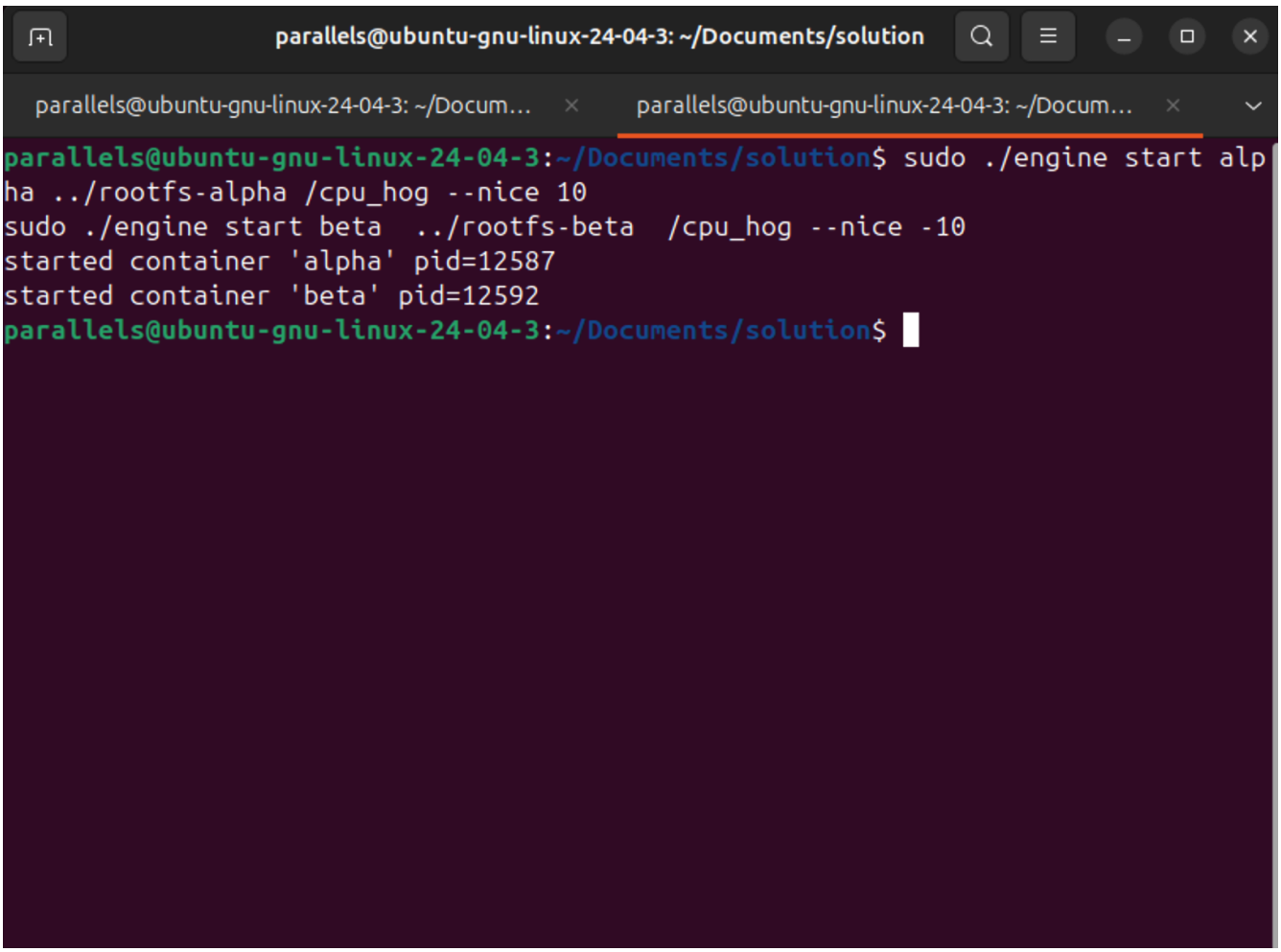

Figure 6.2 - Metadata Tracking (ps output)

Output of the ps command in tabular format showing columns for container name, host PID, state (RUNNING / STOPPED / KILLED), start time, soft limit, hard limit, and log file path for all tracked containers.

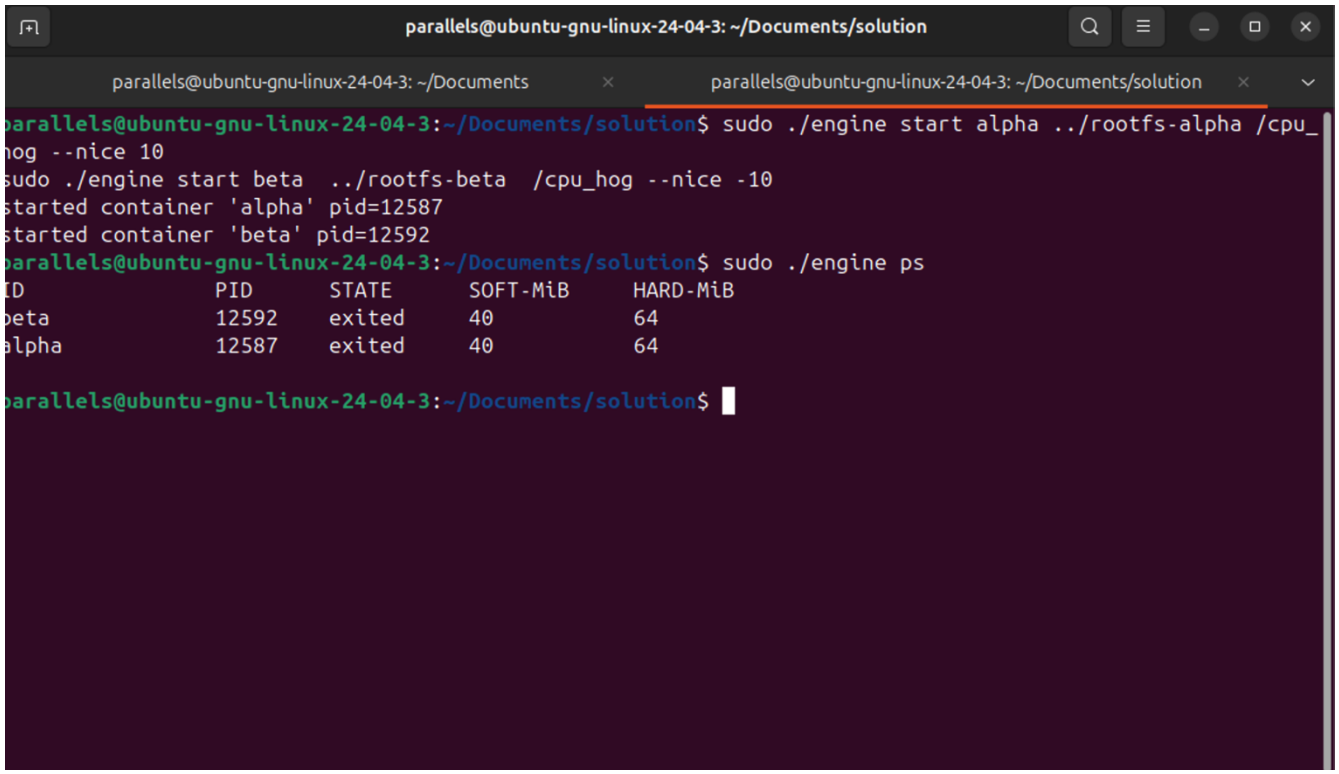

Figure 6.3 - Bounded-Buffer Logging

Contents of a per-container log file captured through the logging pipeline, showing container stdout/stderr output. Producer and consumer debug messages indicate buffer fill level and drain activity, confirming the pipeline is operating correctly.

```
parallels@ubuntu-gnu-linux-24-04-3:~/Documents/solution$ sudo ./engine logs alpha
sudo ./engine logs beta
log: logs/alpha.log
cpu_hog alive elapsed=1 accumulator=12030359534361745805
cpu_hog alive elapsed=2 accumulator=12370390208069341528
cpu_hog alive elapsed=3 accumulator=10700915456118824157
cpu_hog alive elapsed=4 accumulator=8581493861368257634
cpu_hog alive elapsed=5 accumulator=14423220163024446558
cpu_hog alive elapsed=6 accumulator=7522903621616235280
cpu_hog alive elapsed=7 accumulator=12948154911169622561
cpu_hog alive elapsed=8 accumulator=7684855082593590111
cpu_hog alive elapsed=9 accumulator=8816830319670725637
cpu_hog done duration=10 accumulator=8617051033809408054
log: logs/beta.log
cpu_hog alive elapsed=1 accumulator=17551992989963795166
cpu_hog alive elapsed=2 accumulator=6247132102428486562
cpu_hog alive elapsed=3 accumulator=11220375980282871063
cpu_hog alive elapsed=4 accumulator=3551392834851404605
cpu_hog alive elapsed=5 accumulator=12063811023723552234
cpu_hog alive elapsed=6 accumulator=16526078313031585804
cpu_hog alive elapsed=7 accumulator=3918885518492394990
cpu_hog alive elapsed=8 accumulator=16249628764969878583
cpu_hog alive elapsed=9 accumulator=12986202357623867635
cpu_hog done duration=10 accumulator=2255046075735710544
parallels@ubuntu-gnu-linux-24-04-3:~/Documents/solution$
```

Figure 6.4 - CLI and IPC (UNIX Domain Socket)

A CLI command (e.g., stop alpha) issued in one terminal and the supervisor responding in another terminal. This demonstrates that the control message was delivered through the UNIX domain socket and the container state was updated to STOPPED.

```
parallels@ubuntu-gnu-linux-24-04-3: ~/Documents/solution
parallels@ubuntu-gnu-linux-24-04-3: ~/Documents    parallels@ubuntu-gnu-linux-24-04-3: ~/Documents/solution
parallels@ubuntu-gnu-linux-24-04-3:~/Documents/solution$ sudo ./engine start beta  ../rootfs-beta  /cpu_
hog --nice -10
container 'beta' already exists
parallels@ubuntu-gnu-linux-24-04-3:~/Documents/solution$ sudo ./engine ps
ID              PID      STATE      SOFT-MiB     HARD-MiB
beta            12592    exited     40           64
alpha           12587    exited     40           64

parallels@ubuntu-gnu-linux-24-04-3:~/Documents/solution$
```

Figure 6.5 - Soft Limit Warning in dmesg

Output of dmesg showing a kernel-level soft limit warning message for a container whose process has exceeded the configured RSS soft threshold. The message includes the host PID and current RSS value in KB.

```
parallels@ubuntu-gnu-linux-24-04-3: ~/Documents    parallels@ubuntu-gnu-linux-24-04-3: ~/Documents/solution
parallels@ubuntu-gnu-linux-24-04-3:~/Documents/solution$ sudo dmesg | grep container_monitor | tail -n 5

[ 2044.193248] [container_monitor] container=proc_test pid=13850 exited, removing
[ 2265.027248] [container_monitor] Registering container=memtest pid=14720 soft=20971520 hard=41943040
[ 2267.425141] [container_monitor] SOFT LIMIT container=memtest pid=14720 rss=25432064 limit=20971520
[ 2269.473045] [container_monitor] HARD LIMIT container=memtest pid=14720 rss=42209280 limit=41943040
[ 2269.475628] [container_monitor] Unregister request container=memtest pid=14720
parallels@ubuntu-gnu-linux-24-04-3:~/Documents/solution$
```

Figure 6.6 - Hard Limit Enforcement

Output of dmesg showing a hard limit violation message followed by evidence that the process received SIGKILL. The supervisor ps output then shows the container state as KILLED, reflecting the termination reason.

```
parallels@ubuntu-gnu-linux-24-04-3: ~/Documents    parallels@ubuntu-gnu-linux-24-04-3: ~/Documents/solution
parallels@ubuntu-gnu-linux-24-04-3:~/Documents/solution$ sudo dmesg | grep container_monitor | tail -n 5

[ 2044.193248] [container_monitor] container=proc_test pid=13850 exited, removing
[ 2265.027248] [container_monitor] Registering container=memtest pid=14720 soft=20971520 hard=41943040
[ 2267.425141] [container_monitor] SOFT LIMIT container=memtest pid=14720 rss=25432064 limit=20971520
[ 2269.473045] [container_monitor] HARD LIMIT container=memtest pid=14720 rss=42209280 limit=41943040
[ 2269.475628] [container_monitor] Unregister request container=memtest pid=14720
parallels@ubuntu-gnu-linux-24-04-3:~/Documents/solution$
```

Figure 6.7 - Scheduler Experiment Results

Side-by-side terminal output comparing wall-clock completion times for two CPU-bound workloads at different nice values, and a separate comparison between a CPU-bound and an I/O-bound container running concurrently, showing measurable scheduling differences.

parallels@ubuntu-gnu-linux-24-04-3: /media/psf/Home/Downloads/OS-Ja...

parallels@ubuntu-gnu-linux-24-04-3: /media/p... | parallels@ubuntu-gnu-linux-24-04-3: /media/p...

```
op - 23:54:48 up  4:19,  1 user,  load average: 0.07, 0.10, 0.09
asks: 219 total,   1 running, 218 sleeping,   0 stopped,   0 zombie
Cpu(s):  3.2 us,  1.5 sy,  0.0 ni, 95.3 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
iB Mem :   3894.8 total,   1280.4 free,   1561.4 used,   1657.0 buff/cache
iB Swap:   3899.0 total,   2932.3 free,    966.7 used.   2333.4 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 3382 paralle+  20   0 3900148 184812  58880 S   3.6   4.6   6:10.53 gnome-s+
65222 paralle+  20   0  569532  60800  41296 S   1.7   1.5   0:09.23 gnome-t+
 1080 message+  20   0   12428   5152   2680 S   0.7   0.1   0:55.87 dbus-da+
79924 paralle+  20   0 2813324  62092  46768 S   0.7   1.6   0:01.08 gjs
 1658 root      20   0   44928   4240   3396 S   0.3   0.1   0:18.66 cupsd
 3531 paralle+  20   0  810248   7216   5468 S   0.3   0.2   1:23.42 prlcc
 3548 paralle+  20   0  307996   2292   2084 S   0.3   0.1   0:49.74 prldnd
 3852 paralle+  20   0  231028   4828   3940 S   0.3   0.1   0:00.53 dconf-s+
 3947 paralle+  20   0  642636   9068   6388 S   0.3   0.2   0:01.22 xdg-des+
 8396 paralle+  20   0 1619784 154676  31172 S   0.3   3.9   0:38.63 snap-st+
80001 paralle+  20   0   14368   5504   3380 R   0.3   0.1   0:00.29 top
    1 root      20   0   23384  11184   7324 S   0.0   0.3   0:07.67 systemd
    2 root      20   0       0      0      0 S   0.0   0.0   0:00.01 kthreadd
    3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_wo+
    4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker+
    5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker+
    6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker+
```

CHAPTER 7
GITHUB LINK

The complete source code for this project, including engine.c, monitor.c, monitor_ioctl.h, workload programs, Makefile, and README.md, is hosted on GitHub at:

https://github.com/pranav-codes55/OS-Jackfruit

The repository includes step-by-step build and run instructions, annotated output screenshots, engineering analysis addressing the five required areas (isolation mechanisms, supervisor and process lifecycle, IPC and synchronization, memory management and enforcement, and scheduling behavior), design decisions with tradeoffs, and scheduler experiment results with raw data and analysis.

REFERENCES / BIBLIOGRAPHY

[1] Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 10th Edition. Wiley, 2018.

[2] Linux man-pages project. clone(2), chroot(2), mount(2), pipe(2), ioctl(2). Available at: https://man7.org/linux/man-pages/

[3] Robert Love. Linux Kernel Development, 3rd Edition. Addison-Wesley, 2010.

[4] The Linux Documentation Project. The Linux Kernel Module Programming Guide. Available at: https://tldp.org/LDP/lkmpg/

[5] Michael Kerrisk. The Linux Programming Interface. No Starch Press, 2010.

[6] Linux Kernel Source. include/linux/list.h - Kernel linked list API. Available at: https://elixir.bootlin.com/

[7] Alpine Linux Project. Alpine Linux mini root filesystem. Available at: https://alpinelinux.org/downloads/

[8] PES University. UE24CS242B Operating Systems Course Material, Jan-May 2026.

APPENDIX A
DEFINITIONS, ACRONYMS AND ABBREVIATIONS

LKM: Loadable Kernel Module - a compiled object that can be inserted into the running Linux kernel without rebooting.

RSS: Resident Set Size - the portion of a process's memory currently held in RAM, excluding swapped-out pages.

CFS: Completely Fair Scheduler - the default Linux process scheduler since kernel 2.6.23, designed to distribute CPU time fairly using virtual runtime tracking.

IPC: Inter-Process Communication - mechanisms allowing processes to exchange data, including pipes, UNIX domain sockets, shared memory, and FIFOs.

PID: Process Identifier - a unique integer assigned by the kernel to each running process.

UTS Namespace: Unix Timesharing System namespace - a Linux namespace that isolates the hostname and NIS domain name per container.

ioctl: Input/Output Control - a system call enabling user-space programs to send commands to kernel-space drivers or modules.

chroot: Change Root - a system call that changes the apparent root directory of a process and its children to a specified path.

SIGCHLD: Signal sent to a parent process when a child terminates, stops, or resumes.

SIGKILL: A signal that immediately terminates a process and cannot be caught or ignored.

vruntime: Virtual Runtime - the weighted execution time tracked by CFS per task, used to determine which task runs next.

clone(): Linux system call for creating new processes with fine-grained control over which resources (namespaces, file descriptors, memory) are shared with the parent.